

Memory Organisation**(Classification of Memory, Memory Module Design, Cache Memory, Virtual Memory)**Sem: 4th Section: COA-CSE-G1, COA-CSCE-G1

Faculty: Prof. Anil Kumar Swain

Lecture Summary

1. Introduction to Computer memory
2. Classification of Computer Memory
3. Memory module Design
4. Cache Memory
5. Virtual Memory

1 Introduction to Computer Memory

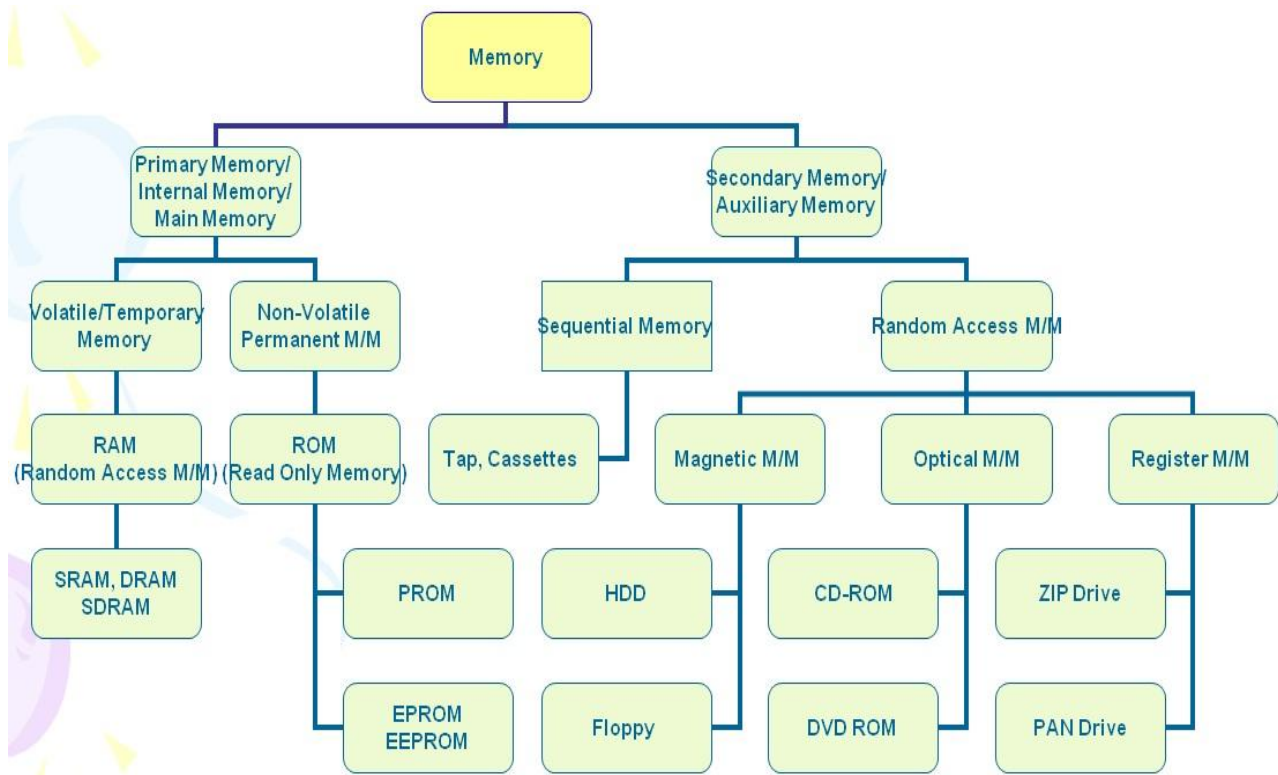
- At the word memory, we think of human memory, and of machine memory.
- Memory is a vital part of both machine and animal. Without it we as humans would not have any consciousness and the ability to create. Animals would not be able to survive.
- In machines, and especially in computers, memory allows the machine to function in various ways, for example software to be run and data to be saved and processed.
- In machines, many different types of electronic memory we encounter in our daily life such as RAM, ROM, Cache, Static RAM, Dynamic RAM, Flash memory, Memory Sticks, Virtual memory, Video memory, BIOS etc.
- Here are just a few examples of the many electronics items, we use everyday that use memory: Cell phones, PDAs, Game consoles, Car radios, VCRs, TVs etc.

1.1 Definition

- Memory is the ability to retain data for a period of time, short or long.
- **The Term Computer Memory is defined as one or more sets of electronic chips that store programs, data, results or any other kind of information, either temporarily or permanently.**
- Memory is the part of a computer that store information. It stores programs, data, results or any other kind of information. It is a storing device.

2 Classification of Memory

- The memory classification as below:



- People in the computer industry commonly use the term "memory" to refer to RAM (Random Access Memory).
- Memory is also known as primary storage, primary memory, main storage, internal storage, main memory, and RAM (Random Access Memory); all these terms are used interchangeably by people in computer circles. Memory is the part of the computer that holds data and instructions for processing.
- Computer memory is extremely important to computer operation. Files and programs are loaded into memory from external media like fixed disks (hard drives) and removable disks (floppies tapes). Memory can be built right into a system board, but it is more typically attached to the system board in the form of a chip or module. Inside these chips are microscopic digital switches which are used to represent binary data.

2.1 Difference Between Primary Memory and Secondary Memory

- **Primary Memory:** Primary memory is the memory that is directly accessed by the CPU to store and retrieve information. Most of the time, primary memory is also referred to as the RAM (Random Access Memory). It is a volatile memory, which loses its data when the power is turned off. Primary memory is directly accessible by the CPU through the address and memory bus and it is constantly accessed by the CPU to get data and instructions. Furthermore, computers contain a ROM (Read Only Memory), which holds instructions that are executed often such as the startup program (BIOS). This is a non volatile memory that retains its data when the power is turned off. Since the main memory is accessed often, it needs to be faster. But they are smaller in size and also costly.
- **Secondary Memory:** Secondary memory is a storage device that is not accessible directly by the CPU and used as a permanent storage device that retains data even after power is turned off. CPU accesses these devices through an input/ output channel and data is first transferred in to the primary memory from the secondary memory before accessing. Usually, hard disk drives and optical storage devices (CDs, DVDs) are used as secondary storage devices in modern

computers. In a secondary storage device, data are organized in to files and directories according to a file system. This also allows to associate additional information with data such as the access permissions, owner, last access time, etc. Furthermore, when the primary memory is filled up, secondary memory is used as a temporary storage for keeping least used data in the primary memory. Secondary memory devices are less costly and larger in size. But they have a large access time.

Difference Between	
Primary Memory	Secondary Memory
1. Primary memory (also referred to as the main memory) is the memory that is directly accessed by the CPU to store and retrieve information. 2. Primary memory is accessed using address and data buses by the CPU. 3. Primary memory is a volatile memory that is it does not retain data when the power is turned off. 4. Primary memory is very fast compared to the secondary memory. 5. These devices have less storage capacity. 6. These devices are expensive.	1. Secondary memory is not accessible directly by the CPU. 2. Secondary memory is accessed using input/output channels. 3. Secondary memory is a non-volatile memory that is it retains data when the power is turned off 4. Secondary memory is very slow as compared to primary memory. 5. These devices have high storage capacity. 6. These devices are cheaper.

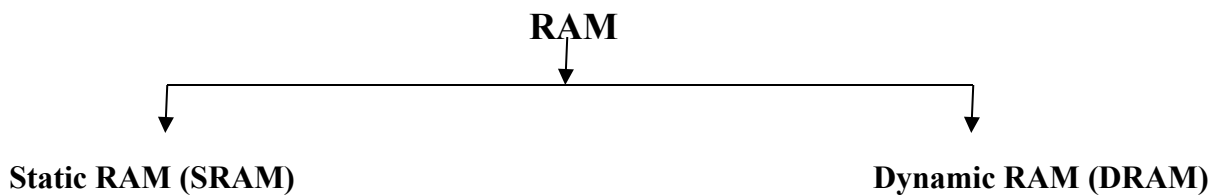
2.2 Primary or Main Memory

- Primary or main memory devices are the ones in which any location can be accessed in any order (or randomly) (not necessarily in sequential order).
 - The time required to access one word (called memory access time) is fixed, independent of the location of the word being accessed.
 - It is a semiconductor memory. The memory contains a large number of semiconductor storage cells, each capable of storing one bit of information. These cells are rarely read or written as individual cells but instead are processed in groups of fixed size called **words**. The memory is organized so that the contents of one word, containing n-bits, can be stored or retrieved in one basic operation.
 - It stores program along with data, which are to be executed. It also stores necessary programs of the system software, which are required to execute the user's program.
 - **The main memory is directly addressed by the CPU** .Therefore All programs to be executed by the CPU need to be stored in main memory. Every such location is uniquely identified (addressed).This is due to the proximity of the main memory devices to the CPU and also due to the size of the main memory devices, which is big compared to the internal memory.
 - Primary memory devices are highly speed devices (but slower than internal CPU registers), located outside the CPU (on the motherboard).
 - There are **two types of primary memory or main memory** devices which process random access property names
 - RAM - Random Access Memory
 - ROM – Read Only Memory
- NB:-** The term Random Access Memory is a misleading, because the contents in both RAM and ROM are randomly accessible(in any order).

- **Difference between**

Random Access Memory (R A M)	Read Only Memory (R O M)
a) Information can be read from a RAM, also can be written into it. b) RAM is volatile memory (It stores information as long as power supply is on. The contents of RAM is lost after powers turned off) c) (Use) It is used for storing the bulk of the programs and data that are subject to change.	a) Information can be read only from a ROM, can't be written into it. Data are written into a ROM when it is manufactured. It is a Read Only Memory. b) ROM is non-volatile memory. (The contents of ROM remains unchanged after power is turned off) c) (Use) It is used for storing programs that are permanent resident in the computer and for tables of constants that don't change in value.

2.3 Types of RAM

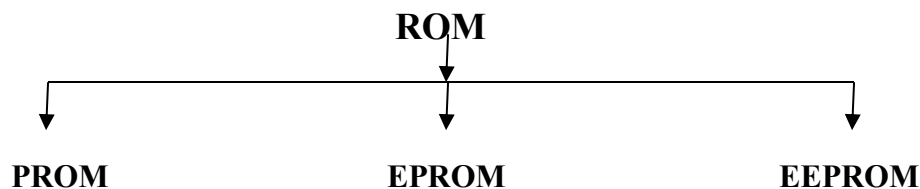


- **Difference Between**

Static RAM (SRAM)	Dynamic RAM (DRAM)
1. SRAM stands for Static RAM (Memories that consists of circuits capable of returning their state as long as power is applied are known as static memories). 2. SRAM uses conventional flip flop circuit consisting of two cross coupled invertors to store binary bit 0 or 1. 3. Six transistors are needed to form a memory cell of SRAM. 4. As six transistors are required to form a memory cell , SRAM contains less memory cells per unit area. Hence its packing density is low, hence smaller in size . 5. Refreshing circuit is not required.	1. DRAM stands for Dynamic RAM (Such memory cells donot return their state indefinitely, hence they are called Dynamic RAMs). 2. DRAM stores information in a dynamic memory cell in the form of a charge on a capacitor. 3. It consists of one transistor (MOSFET) and capacitor for each cell. 4. As one transistor is required to form a memory cell , DRAM contains more memory cells per unit area . Hence its packing density is high, hence larger in size. 5. A DRAM stores information in the form of charge on a capacitor , which leaks away in a very short time (that means a a DRAM losses its stored information in a few milliseconds even though its power supply is on). Since the cell is required to store information for a much longer time its contents must be periodically refreshed by

6. The major advantage of CMOS RAM is their low power consumption because current flows in the cell only when the cell is being accessed. 7. Cost is more due to it consists more transistors. 8. Its access time is less hence SRAM is a faster memory. 9. (Use) SRAMs are used in application where speed is of critical concern. SRAM is used as cache memory. 10. Typical access time = a few nano second (10 nano second). Typical sizes are available.	storing the capacitor charged to its full value (usually every 2 milliseconds it requires a refreshing and control circuitry). 6. High power consumption. 7. Cost is less. 8. Its access time is more than SRAM, hence DRAM is slower memory than SRAM. 9. (Use) DRAMs are used in applications where cost and size are of critical concern. DRAM is used as main memory. 10. Typical access time = 50 nano second. Typical sizes available = 1 M 256 Mbits. A 64 Mbit chip may be organized as 16Mx4, 8Mx8, 4Mx16.
---	--

2.4 Variations of ROM



- a) **ROM – (Read Only Memory)** Rom portion of main memory is needed for storing an initial program called bootstrap loader(it is a program).When power is turned on the hardware of the computer sets the program counter to the first address of the bootstrap loader. The bootstrap loader loads a portion of the operating system from disk to main memory and control is then transferred to the operating system which prepares the computer for general use.
- ROM is similar to RAM, but is non-volatile in nature (i.e the content of these remain as it is even in the absence of electrical power)
 - Usually these are used to store small amount of information (that are permanent resident in the computer) for quick reference by the CPU.
 - ROM is a read only memory which means information can be read from a ROM chip, but fresh information can't be written into it by the user under normal circumstances.
 - Data are written into a ROM when it is manufactured.
 - ROM is also considered to be an example of firmware.
 - **Firmware** : Firmware is a hardware device with software embedded in it.It allows reading and executing the software embedded in it. It allows reading and executing the software but does n't allow modification

b) **PROM** -Programmable ROM

- PROM is similar to ROM except it allow the data to be loaded by the user, thus providing programmable ROM.
- Before it is programmed, the memory contains all zeros. The user can insert 1s at the required locations by burning out the fuses at these locations using high current pulses.
- PROM provide a faster and considerably less expensive approach for storing fixed programs and data when high volumes of ROMs are produced, because they can be directly programmed by the user.
- Once PROM stored information ,it behaves line a ROM.(PROM are only once programmable)

c) **EPROM** - Erasable and Programmable ROM

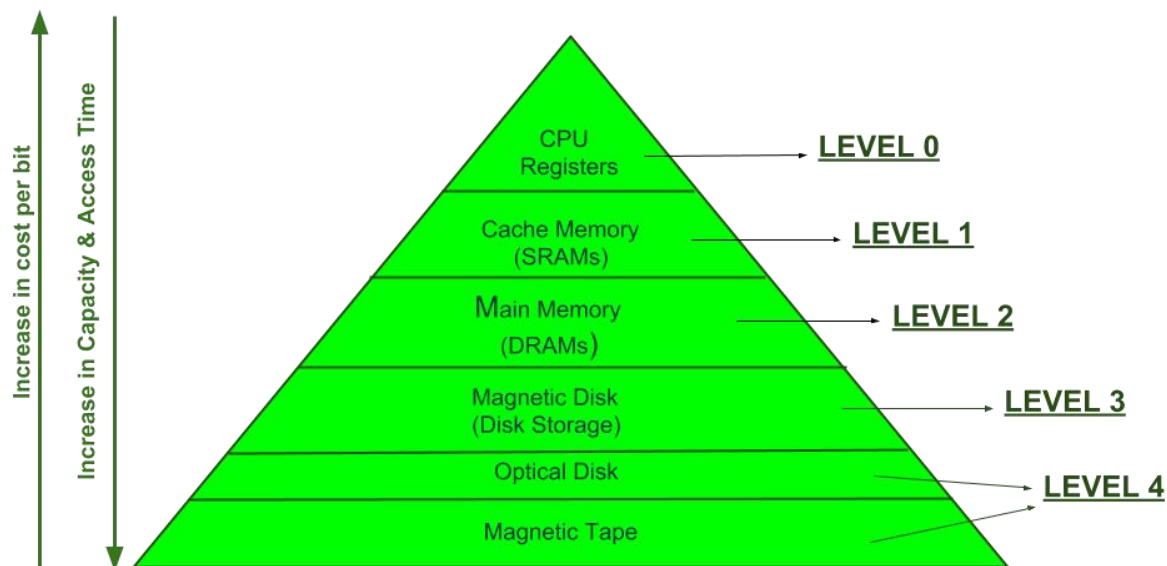
- The type of ROM chip which allows the stored data to be erased and new data to be loaded (it can be reprogrammed more than once),is called EPROM.
- Erasing can be done by exposing the EPROM chip to high intensity short wave ultraviolet for about 20 minites.
- To fascilitate exposure of ultraviolet light, the EPROM chips are pached in a pache which has transparent windows. The disadvantage of this technique of eraser is that the EPROM chip has to be taken out of the system board and placed in an EPROM erasers chamber.
- (ADV) EPROMs are chip and reliable. Hence, they are widely used. EPROMs are used to store programs and data or any kind of information which are permanent, but need updating.

d) **EEPROM** - Electrically Erasable and Programmable ROM

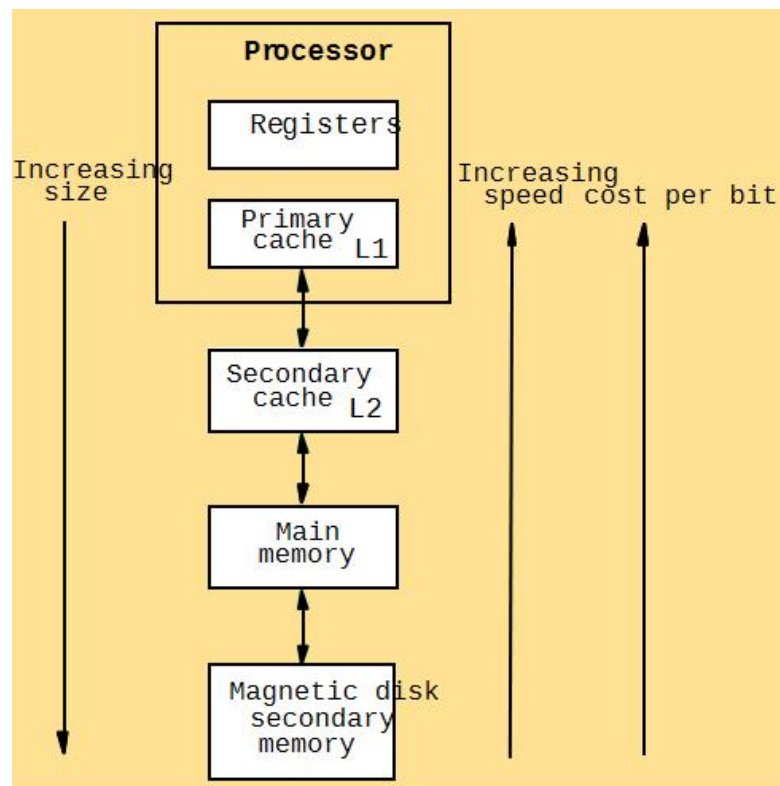
- EEPROMs are electrically erasable PROMS.They are also called EAROM(Electrically Alterable ROM).
- A significant disadvantage of EPROMs is that a chip must physically removed from the circuit for reprogramming and its entire content are erased by the ultraviolet light.
- It is possible to implement another version of erasable PROMs that can be both programmed and erased electrically. Such chips are called EEPROM OR E²PROMS do not have to be removed for erase. Moreover it is possible to erase cell contents selectively as EPROM is byte erasable.
- The only dis advantage of EPROMs is that different voltages are needed for erasing ,writing and reading the stored data.
A pulse 21 volts for writing and erasing 5 volts for reading.
- It uses complex memory cell structure and therefore it has low density,it has also hish cost and lower reliability.
- It is used in certain applications where byte-alterability is needed.

2.5 Memory Hierarchy

- In the Computer System Design, Memory Hierarchy is an enhancement to organize the memory such that it can minimize the access time. The Memory Hierarchy was developed based on a program behavior known as locality of references. The figure below clearly demonstrates the different levels of memory hierarchy :
- Fastest access is to the data held in processor registers. Registers are at the top of the memory hierarchy.
- Relatively small amount of memory that can be implemented on the processor chip. This is processor cache. Two levels of cache. Level 1 (L1) cache is on the processor chip. Level 2 (L2) cache is in between main memory and processor.
- Next level is main memory, implemented as SIMMs. Much larger, but much slower than cache memory.
- Next level is magnetic disks. Huge amount of inexpensive storage. Speed of memory access is critical, the idea is to bring instructions and data that will be used in the near future as close to the processor as possible.



MEMORY HIERARCHY DESIGN



- This **Memory Hierarchy Design** is divided into 2 main types:
 - a) **External Memory or Secondary Memory** –
Comprising of Magnetic Disk, Optical Disk, Magnetic Tape i.e. peripheral storage devices which are accessible by the processor via I/O Module. External or Auxillary memory access time is generally 1000 times that of the main memory, hence it is at the bottom of the hierarchy.
 - b) **Internal Memory or Primary Memory** –
Comprising of Main Memory, Cache Memory & CPU registers. This is directly accessible by the processor. The main memory occupies the central position because it is equipped to communicate directly with the CPU and with auxiliary memory devices through Input/output processor (I/O).
- **Characteristics of Memory Hierarchy Design**
 - i. **Capacity:**
It is the global volume of information the memory can store. As we move from top to bottom in the Hierarchy, the capacity increases.
 - ii. **Access Time:**
It is the time interval between the read/write request and the availability of the data. As we move from top to bottom in the Hierarchy, the access time increases.
 - iii. **Performance:**
Earlier when the computer system was designed without Memory Hierarchy design, the speed gap increases between the CPU registers and Main Memory due to large difference in access time. This results in lower performance of the system and thus, enhancement was required. This enhancement was made in the form of Memory Hierarchy Design because of which the performance of the system increases. One of the most significant ways to

increase system performance is minimizing how far down the memory hierarchy one has to go to manipulate data.

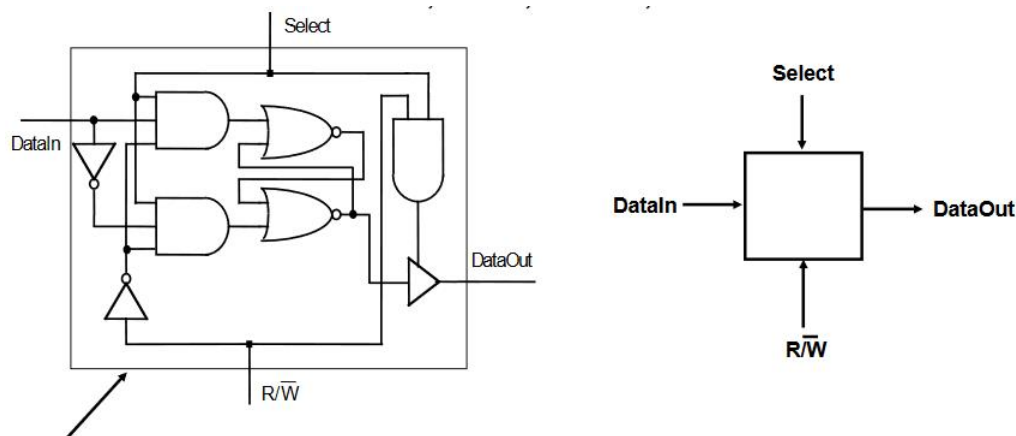
iv. **Cost per bit:**

As we move from bottom to top in the Hierarchy, the cost per bit increases i.e. Internal Memory is costlier than External Memory.

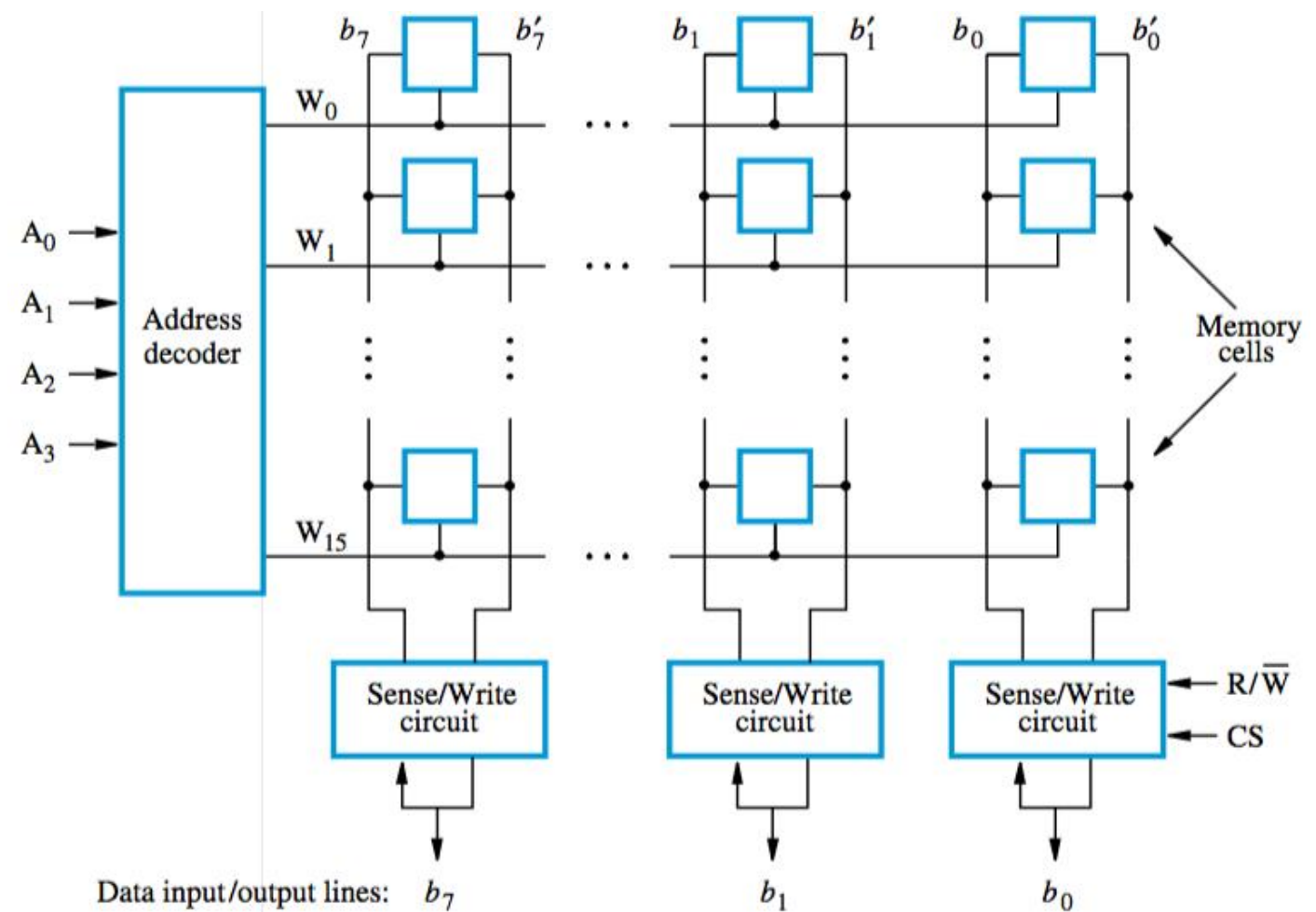
3 Semiconductor RAM Memory Modules

3.1 Internal Organisation of Memory Chips

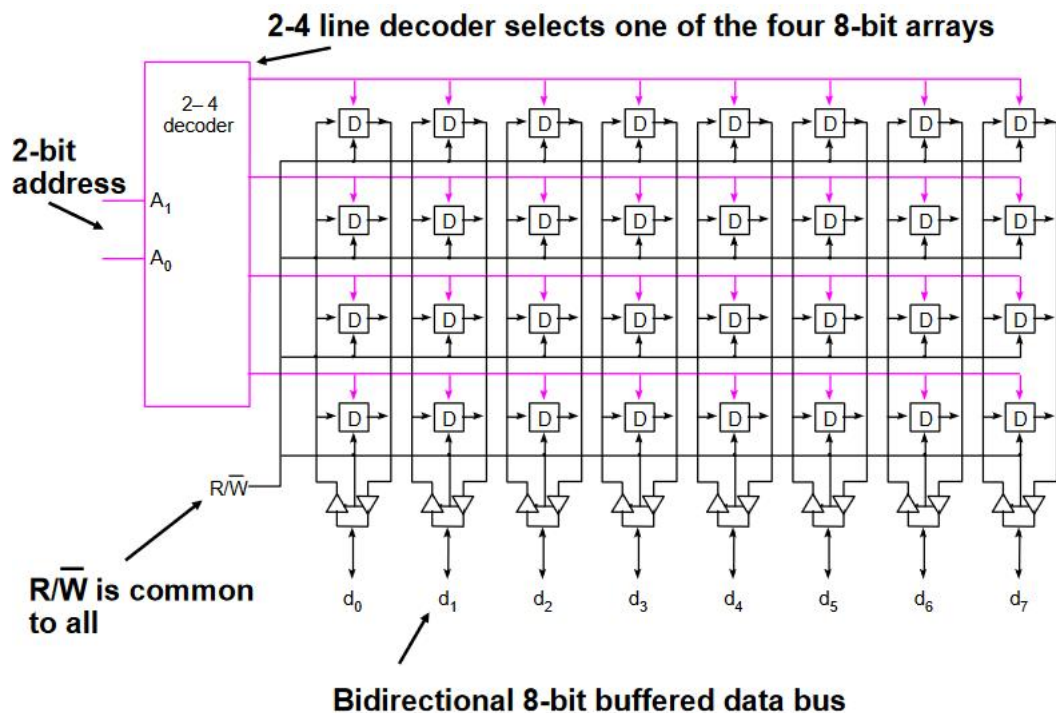
- A memory cell is capable of storing 1 bit of information. Regardless of the technology, all RAM memory cells must provide these four function. Select, DataIn, DataOut, and $R/\bar{W}$.
- A number of memory cells are organised in the form of a matrix to form the memory chip.
- Each row of the cells constitutes a memory words and all cells of a row are connected to a common line referred to as a word line, which is driven by the address decoder on the chip. Thus $W \times b$ memory has w words, each word having 'b' number of bits.
- The cells in each column are connected to a **sense/write circuit** by **two bit lines**.
 - i. **R/W** : This input specifies the required operation (Read or Write). During a Read operation, these circuits sense or read the information stored in the cells, selected by a word lines and transmit this information to the output data lines. During a Write operation, these circuits receive input information and store in the cells of the selected word.
 - ii. **CS (Chip Select)**: This input selects a chip in a multi chip memory system.



This "static" RAM cell is unrealistic in practice, but it is functionally correct.
(Sample Memory Cell)

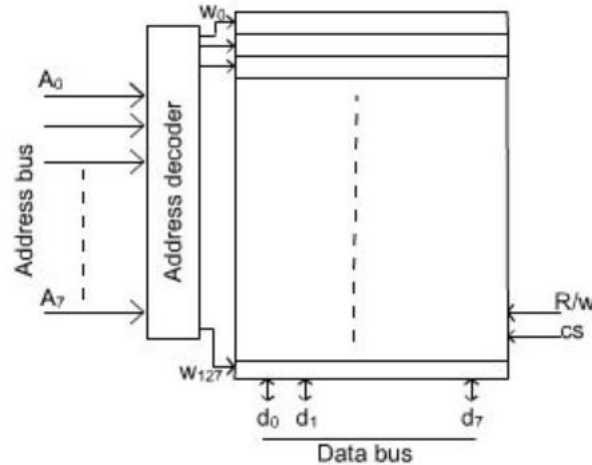


(Organisation of bit cells in a memory chip)
(A 16 x 8 2-D Memory Cell Array)

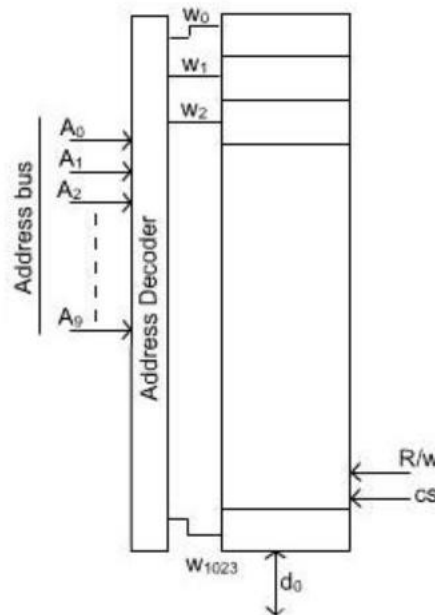


(A 4 x 8 2-D Memory Cell Array)

- A number of memory cells are organised in a memory chips.
- Consider a memory unit that has 1K (1024) memory cells that is **128 x 8 memory chip**: If it is organised as a 128 x 8 memory chips, then it has got 128 memory words of size 8 bits. So the size of the data bus is 8 bits and the address bus is 7 bits ($2^7=128$). The storage organisation of the 128 x 8 memory chip is shown below

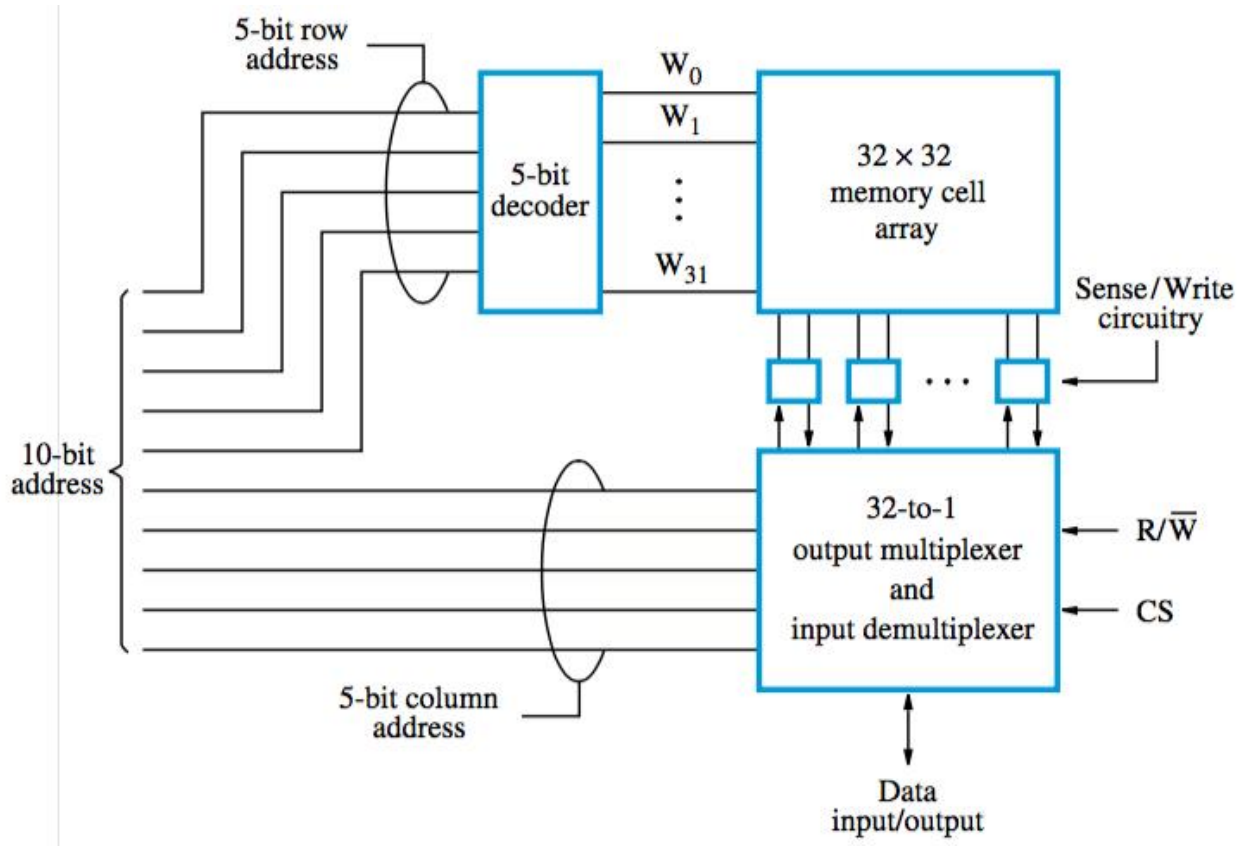


- Consider a memory unit that has 1K (1024) memory cells that is **1024 x 1 memory chip**: If it is organised as a 1024 x 1 memory chips, then it has got 1024 memory words of size 1 bit. So the size of the data bus is 1 bit and the address bus is 10 bits ($2^{10}=1024$). The storage organisation of the 1024x 1 memory chip is shown below (one way).



The storage organisation of the 1024 x 1 memory chip is shown below (another way).

- In this case several memory words are organised in one row.
- Address bus is divided into two groups.
 - i. Gr-1: To form the row address
 - ii. Gr-2: To form the column address
- 10-bit address bus, divide each group 5 bits.



- A row address selects a row of 32 cells, all of which are accessed in parallel. However, according to the column address, only one of these cells is connected to the external data line via the input output multiplexers. The arrangements for row address and column address decoders is shown in the figure above.

3.2 Structure of Larger Memories

- Memory circuits (i.e. specially organisation of smaller memories) may be implemented on a single chip.
- However such memory chips may be interconnected to form a much larger memory.

3.3 Solved Problems (Memory Module Design)

Q1. Consider a memory consisting of 2M (2,097,152) words of 32 bits each. Implement this memory through a block diagram using 512K x 8 static memory chips. In other words, Give a block diagram for a 2M x 32 memory using 512K x 8 memory chips. (Ref. Carl Hamacher CO book 5th Ed, example, page no.305-306)

Solution

Available chip size = 512K x 8

Required memory size = 2M x 32

Implement

Column = $32/8=4$

Row = $2M/512K=2^{21}/2^{19}=4$

Decoder => 2-to-4 decoder

Number of chip will be required = $4 \times 4=16$

2M x 32 => 2M words and 32bits address bus

2M = $2 \cdot 2^{20}=2^{21}$ => 21 bits address bus

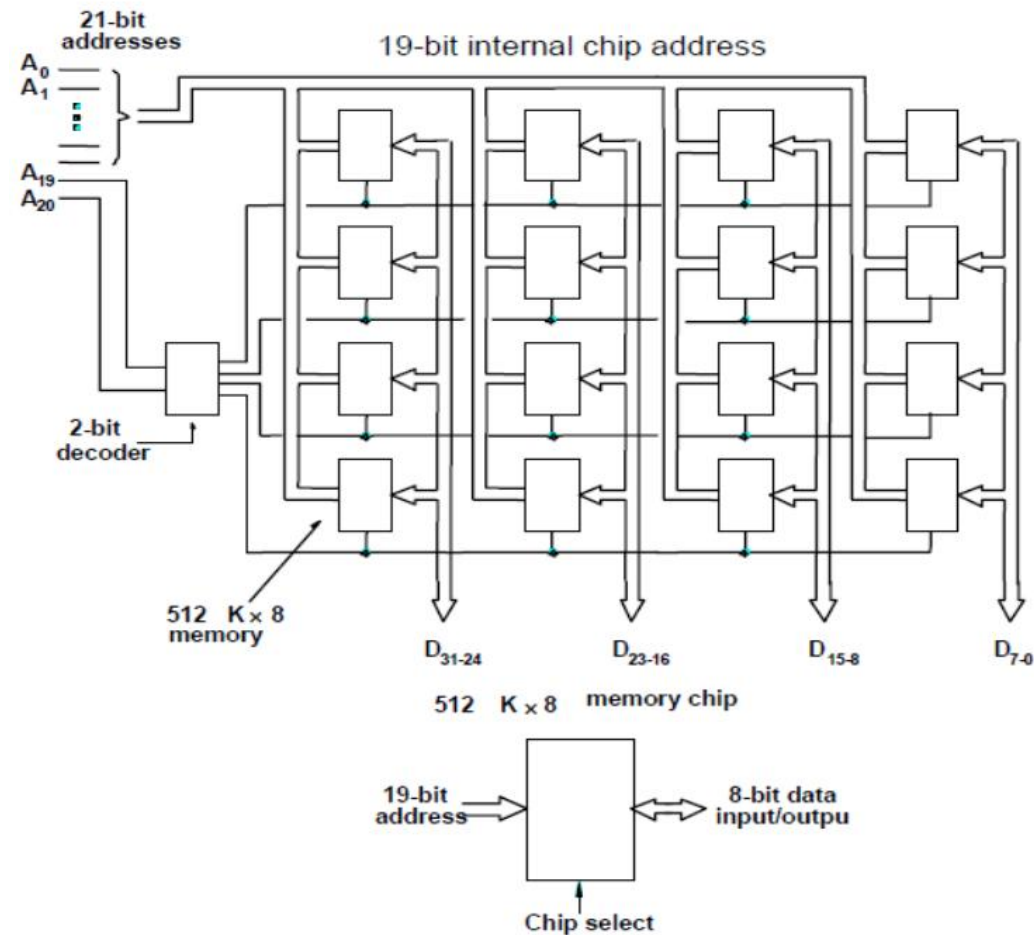
$512K \times 8 \Rightarrow 512K$ words and 8 bits address bus

$512K = 2^9 \cdot 2^{10} = 2^{19} \Rightarrow 19$ bits address bus

As each row of the cells constitute a memory words (32 bits in this case), so with 8 bits (i.e. 8 cells) available chips, we require 4 chips in each row.

As each chip has 512K words and we require 2M words, so there will be 4 ($2M/512K$) rows ($4=2^2 \Rightarrow 2$ -to-4 decoder required) and each column in the figure consists of four chips.

21 address bits are needed to select a 32 bit word in this memory. The higher order 2 bits of the address are decoded to determine which of the four chips select control signals should be activated and the remaining 19 address bits are used to access specific byte locations inside each chip of the selected row.



(Organisation of a 2M x 32 memory module using 512K x 8 static memory chips)

Q2. Give a block diagram for a 8M x 32 memory using 512K x 8 memory chips. (Ref. Carl Hamacher CO-BOOK-Exercise-5.1 problem)

Solution

$8M \times 32 \Rightarrow 8M$ words and 32bits address bus

$8M = 2^3 \cdot 2^{20} = 2^{23} \Rightarrow 23$ bits address bus

$512K \times 8 \Rightarrow 512K$ words and 8 bits address bus

$512K = 2^9 \cdot 2^{10} = 2^{19} \Rightarrow 19$ bits address bus

As each row of the cells constitute a memory words (32 bits in this case), so with 8 bits (i.e. 8 cells) available chips, we require **4 chips in each row**.

As each chip has 512K words and we require 8M words, so there will be $8M/512K = 16$ rows ($16 = 2^4 \Rightarrow$ 4-to-16 decoder required) and each column in the figure consists of four chips.

23 address bits are needed to select a 32 bit word in this memory. The higher order 4 bits of the address are decoded to determine which of the 16 chips select control signals should be activated and the remaining 19 address bits are used to access specific byte locations inside each chip of the selected row.

Block Diagram

Similar to the above diagram except it requires 16 rows. Address lines A_{18-0} are connected to all chips. Address lines A_{22-19} are connected to a 4-bit decoder to select one of the 16 rows.

- Q3. Design the $2M \times 32$ memory using $1024K \times 8$ RAM chip. How many memory chip will be required?**

Solution

$$2M = 2 \cdot 2^{20} = 2^{21}$$

$$1024K = 2^{10} \cdot 2^{10} = 2^{20}$$

$$\text{Column} = 32/8 = 4$$

$$\text{Row} = 2M/1024K = 2$$

2 row ($2MB/1024K$) and 4 column ($32/8$) array

1 bit row decoder (1-to-2 decoder) $\Rightarrow A^{20}$ to A^{21}

20 bit internal chip address (A_0 to A_{19})

Memory chip requirement = $2 \times 4 = 8$

- Q4. How many RAM chips of size $128K \times 16$ are required to provide a capacity of main memory equal to $4M \times 32$? (Ref.KIIT/CSE/END-COA-2015 REG QUESTION)**

Solution

Answer is

$$\text{No. Of rows} = 4M/128K = (2^2 \cdot 2^{20}) / (2^7 \cdot 2^{10}) = 2^{22}/2^{17} = 2^5 = 32$$

$$\text{No. Of Columns in each row} = 32/16 = 2$$

So the number of memory modules required will be $32 \times 2 = 64$

- Q5. To build a $1G \times 16$ memory system, the number of $256M \times 8$ memory modules required will be**

Solution

Correct answer is 8

$$1G = 2^{30}$$

$$256M = 2^8 \cdot 2^{20} = 2^{28}$$

$$\text{Row} = 1G/256M = 4$$

$$\text{Column} = 16/8 = 2$$

$$\text{Memory} = 4 \times 2 = 8$$

There will be 4 ($1G/256M$) rows and each row there will be 2 ($16/8$) columns. The **$256M \times 8$** chips will be arranged in (4×2) array.

$4 \times 256M = 1G$, and $2 \times 8 = 16$. Hence the number of memory modules required will be $4 \times 2 = 8$.

- Q6. For a 2K x 16 memory system that uses a decoder to select a word, the total number of external connections to the memory system (including address, data, control, and power signals) will be at least**

Solution

Correct answer is 31.

There will be 11 lines for the address ($2^{11}=2K$), 16 lines for the data, 2 lines for the power supply, and 2 RD and WR lines. The total is 31.

- Q7. Assume that a 1G x 1 DRAM memory cell array is organized as 1M rows and 1K columns. The number of address bits required to select a row and a column will be _____.**

Solution

Correct answer is 20 bits for row and 10bits for column

Since there are 1M rows, the number of row address lines will be 20, as $2^{20}=1M$. Also, for selecting one of the 1K columns, the number of column address lines will be 10, as $2^{10}=1K$.

- Q8. Do as directed.**

- How many 128 x 8 RAM chips are needed to provide a memory capacity of 2048 bytes?**
- How many lines of the address bus must be used to access 2048 bytes of memory? How many of these lines will be common to all chips?**
- How many lines must be decoded for chip select? Specify the size of the decoders.**

Solution

a. $128 \times 8 \text{ RAM} \Rightarrow 128 \text{ Bytes (8bits)} / \text{chip} \Rightarrow \text{Number of chips} = 2048 / 128 = 16$

b. $2048 = 2^d$, Where d is number of address lines $\Rightarrow d = 11$ We have 16 chips $\Rightarrow 4$ bits for chip select Number of common address lines is $11 - 4 = 7$

Summary: 4 bits (MSB) to select the correct chip and 7 bits (LSB) to select the memory location inside the selected chip.

c. We need 4 x 16 line decoder.

Other Way

Required memory module: $2048 \text{ byts} = 2048 \times 8$

Available memory chips of size = 128×8

Row = $2048/128=16$

Column = $8/8=1$

So total chips = $16 \times 1 = 16$

Address bus to access 2048 bytes of memory = $2^{11} \Rightarrow 11$ bits.

Number of lines of address bus that are common to all chips = $128 = 2^7$

Number of lines must be decoded for chip select = $16 = 2^4 \Rightarrow 4$ bits. So the size of the decoders is (4 x 16).

- Q9. A computer uses RAM chips of 1024 x 1 capacity. How many chips are needed and how should their address lines be connected to provide a memory capacity of 1024 bytes? How many chips are needed to provide a memory capacity of 16K bytes? Explain in words how the chips are to be connected to the address bus.**

Solution

Memory size is 1024 bytes = $8 \times 1024 \times 1 \text{ RAM} \Rightarrow 8$ chips

All has same address lines and output is one bit from every chip.

16K bytes = $16 \times 1024 \times 8 \Rightarrow 128$ chips.

16 groups of 8 chips which have same address/chip select lines.

First chip of every group connected to first bit of output. Second chips to second bit of output and so on.

Q10. What is the function of the chip select line (CS') in a memory chip?

- a) Power supply is applied to the chip when CS' is activated.
- b) The data bus is put in the high impedance state when CS' is deactivated.
- c) It prevents two or more subsystems from using the memory simultaneously.
- d) None of the above.

Solution

Correct answer is (b).

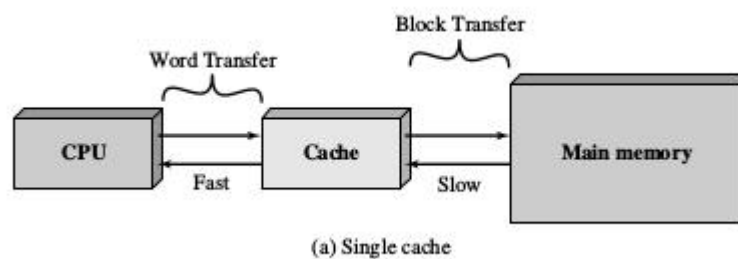
When a memory system consists of a number of memory chips, typically the data lines of the memory chips are connected together. For example, all the D0 data lines will be connected, all the D1 data lines will be connected, and so on. When a number of data output lines are tied together, exactly one output must be active at a time and all the others must be in the high impedance state. The chip select signal forces all the data outputs of a chip to go into the high impedance state if it is deactivated.

4 Cache Memory

- At the word memory, we think of human memory, and of machine memory.

4.1 Principles of Cache Memory/What is cache memory?

- Cache memory is a smaller and faster memory than RAM that acts as a buffer between RAM and the CPU.
- Cache memory is used to speed up and synchronizing with high-speed CPU. In other words, It stores copies of the data or contents of the main memory that are used frequently by CPU into cache memory so that the processor can easily access that data in a shorter time. (It is used to reduce the average time to access data from the Main memory).



- It is also more expensive than main memory, as it is a more complex chip that yields higher performance. What it sacrifices in size and price, it makes up for in speed. Cache memory operates between 10 to 100 times faster than RAM, requiring only a few nanoseconds to respond to a CPU request.
- The name of the actual hardware that is used for **cache memory** is high-speed static random access memory (**SRAM**). The name of the hardware that is used in a computer's **main memory** is dynamic random access memory (**DRAM**).

4.2 Cache Performance:

- The **basic operation of a cache memory** is as follows:
 - ❖ When the CPU needs to access main memory (need to read or write a location in main memory), it first checks for a corresponding entry in the cache. If the processor finds that memory location in the cache, a **cache hit** has occurred and data is read from cache.
 - ❖ If the processor does not find that memory location in the cache, a **cache miss** has occurred. For a cache miss, the CPU moves into the main memory and data/instructions are read from the main memory and sends the same to the cache that allocates a new entry and then the request of CPU is fulfilled from the contents of the cache.
- The **performance of cache memory** is frequently measured in terms of a quantity called **Hit ratio or Hit rate**.

$$\text{Hit ratio} = \text{Hit rate} = \text{hit} / (\text{hit} + \text{miss}) = \text{no. of hits/total accesses}$$

Where,

hit is a successful access to data in a cache. When the CPU refers to memory and finds the word in cache, it is said to produce a **hit**.

miss is the unsuccessful access to data in a cache. If the word is not found in the cache, it is in main memory and it counts as a **miss**.

The ratio of the number of hits divided by the total CPU references to memory (hits plus misses) is the **hit ratio**.

- Similarly, **miss rate** is defined as the ratio of the number of misses divided by the total CPU references to memory (hits plus misses).

$$\text{Miss ratio} = \text{Miss rate} = \text{miss} / (\text{hit} + \text{miss}) = \text{no. of misses/total accesses}$$

- **Miss penalty:** After a miss, the extra time needed to bring the desired information into the cache is called as miss penalty.
- **Average Memory Access Time (AMAT):**
A single-level cache is pretty easy to model mathematically. It is the average to access memory considering both hits and misses in the cache.

The cache performance or Average Memory Access Time (AMAT) experienced by the processor with single cache system is

$$\text{AMAT} = t_{\text{avg}} = hC + (1-h)M$$

Where,

h is the hit rate in the cache

C is the time to access information in the cache

M is the time to access information in the main memory

Explanation

$$\text{AMAT} = t_{\text{avg}} = \text{Hit Rate} * \text{Hit Time} + \text{Miss Rate} * \text{Miss Penalty}$$

Or

$$\text{AMAT} = t_{\text{avg}} = \text{HR} * \text{HT} + \text{MR} * \text{MP}$$

$$= HR*HT + (1-HR)*MP$$

$$= h*C + (1-h)*M$$

- In a two-level cache,

The cache performance or Average Memory Access Time (AMAT) experienced by the processor with two-level cache system is

$$AMAT = t_{avg} = h_1C_1 + (1-h_1)h_2C_2 + (1-h_1)(1-h_2)M$$

Where,

h_1 is the hit rate in the L_1 cache

h_2 is the hit rate in the L_2 cache

C_1 is the time to access information in the L_1 cache

C_2 is the time to access information in the L_2 cache

M is the time to access information in the main memory

Explanation

$AMAT = HR_1*HT_1 + MR_1*MP_1$, where

$MP_1 = HR_2*HT_2 + MR_2*MP$

Now,

$$AMAT = t_{avg} = HR_1*HT_1 + MR_1*MP_1$$

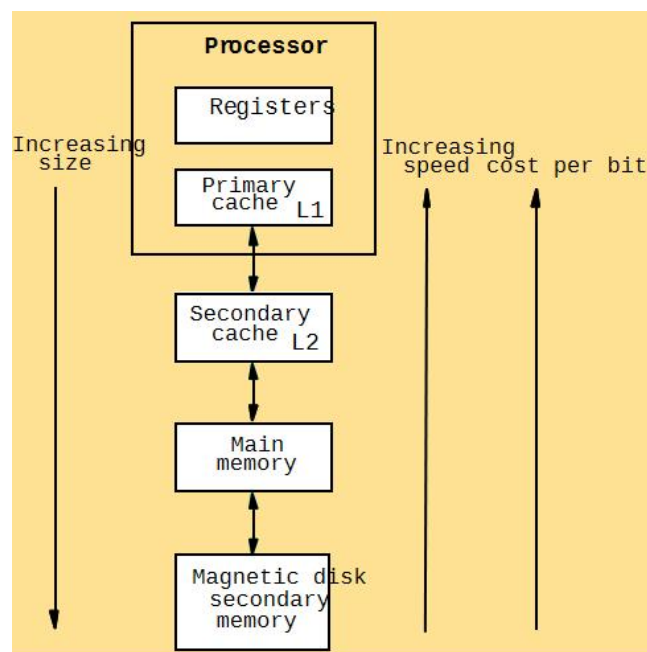
$$= HR_1*HT_1 + MR_1*(HR_2*HT_2 + MR_2*MP)$$

$$= HR_1*HT_1 + MR_1*HR_2*HT_2 + MR_1*MR_2*MP$$

$$= HR_1*HT_1 + (1-HR_1)*HR_2*HT_2 + (1-HR_1)*(1-HR_2)*MP$$

$$= h_1C_1 + (1-h_1)h_2C_2 + (1-h_1)(1-h_2)M$$

- We can improve Cache performance using higher cache block size, higher associativity, reduce miss rate, reduce miss penalty, and reduce the time to hit in the cache.



(Two level cache Memory Hierarchy)

4.3 Solved Problems (Cache Performance)

Q1. Assume that a read request takes 50 nsec on a cache miss and 5 nsec on a cache hit. While running a program, it is observed that 80% of the processor's read requests result in a cache hit. The average read access time is _____ nsec

Solution

Answer: 14

Explanation

Cache Memory Hit Time=HT=C=5ns

Cache Miss Penalty=Main memory access time=MP=M=50ns

Cache Memory Hit Rate=HR=h=80%=0.8 => Cache Memory Miss Rate=MR=0.2

Then,

Average time to read=AMAT=HR*HT + MR*MP=0.8*5 + 0.2*50=14nsec

or

Average read time to read= 0.80 x 5 + (1 -0.80) x 50 = 14 nsec

Q2. Consider a system with 2 level caches. Access times of Level 1 cache, Level 2 cache and main memory are 1 ns, 10ns, and 500 ns, respectively. The hit rates of Level 1 and Level 2 caches are 0.8 and 0.9, respectively. What is the average access time of the system ignoring the search time within the cache? (Ref.GATE-IT-2004/Q.12)

Solution

Answer: 12.6 ns

Explanation

First, the system will look in cache1. If it is not found in cache1, then cache2 and then further in main memory (if not in cache2 also).

The average access time would take into consideration success in cache1, failure in cache1 but success in cache2, failure in both the caches and success in main memory.

Data Given,

Access times of Level 1 cache, Level 2 cache and main memory are 1 ns, 10ns, and 500 ns, respectively => **C1=1 ns, C2=10 ns, M=500 ns**

The hit rates of Level 1 and Level 2 caches are 0.8 and 0.9, respectively=>

$h_1=0.8, h_2=0.9$

By using the formulae for AMAT experienced by the processor with two-level cache system is

$$\begin{aligned} \text{AMAT} = t_{\text{avg}} &= h_1 C_1 + (1-h_1)h_2 C_2 + (1-h_1)(1-h_2)M \\ &= 0.8 \times 1 + (1-0.8) \times 0.9 \times 10 + (1-0.8) \times (1-0.9) \times 500 \\ &= 0.8 + 0.2 \times 0.9 \times 10 + 0.2 \times 0.1 \times 500 \\ &= 0.8 + 1.8 + 10 = 12.6 \text{ ns.} \end{aligned}$$

4.4 Locality of reference

- Since size of cache memory is less as compared to main memory. So to check which part of main memory should be given priority and loaded in cache is decided based on locality of reference.
- **Principles of Locality: Program access a relatively small portion of the address space at any instant of time. •Example: 90% of time in 10% of the code.**



- **Types of Locality of reference**

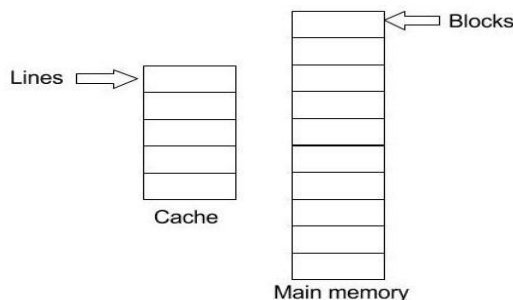
- a) **Temporal Locality (Locality in Time) of reference:** If at one point a particular memory location is referenced, then it is likely that the same location will be referenced again in the near future.
- b) **Spatial Locality (Locality in Space) of reference:** If a particular storage location is referenced at a particular time, then it is likely that nearby memory locations will be referenced in the near future.

4.5 Types of Cache

- **Primary Cache:** A primary cache is always located on the processor chip. This cache is small and its access time is comparable to that of processor registers. It is referred to as the level 1 (L1) cache.
- **Secondary Cache:** Secondary cache is placed between the primary cache and the rest of the memory. It is referred to as the level 2 (L2) cache. Often, the Level 2 cache is also housed on the processor chip.

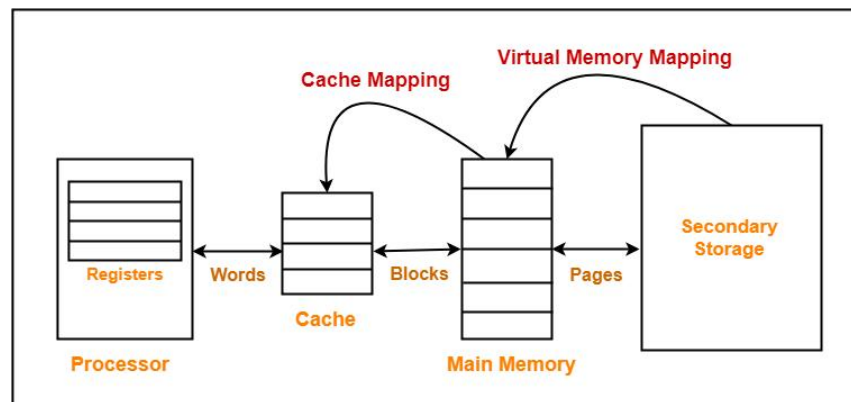
4.6 Cache Mapping

- In order to implement the cache principle, we need to know the following terms:
 - **Main Memory Blocks(or Blocks):** Main memory is divided into equal size partitions are called as main memory blocks or simply blocks. Let the main memory has 64K words. If we keep 16 words per blocks, then total number of main memory blocks= $64K/16=24K$ blocks
 - **Cache Blocks or Cache Lines (or Lines):** Cache memory is divided into partitions having same size as that of main memory blocks called as Cache Blocks or Cache Lines or Lines. Let the cache memory has 2K words. As main memory block size is fixed, we can take the same size for the cache block that is 16 words per cache block. Now, total number of cache blocks= $2K/16=(2.2^{10})/2^4=2^{11}/2^4=2^7=128$ lines



- (Recap) When CPU attempts to read a word from main memory, check is made to determine if the word is in cache. If the word is found in cache, then word is delivered from cache. This is called a **cache hit**. If word is not there in cache then, this is called a **cache miss**. In case of cache miss, a block of main memory consisting some word along with that word, is read into cache and the required word is delivered to CPU. This is called “**Principle of Locality of Reference**”.

- During a cache miss if there are no empty blocks in the cache, then some replacement algorithms such as FIFO, LRU, LFU, etc. are used to replace a cache block by main memory block.
- **Defⁿ:** The process of transferring (more precisely copying) the content of main memory (data or instructions) to cache memory is called as **cache mapping** in case of a cache miss. In other words, cache mapping defines how a block from the main memory is mapped to the cache memory block in case of a cache miss.
- The following diagram illustrates the mapping process.



- **Types of Cache Mapping:** The different Cache mapping technique are as follows:-
 - a) Direct mapping
 - b) Associative mapping
 - c) Set Associative mapping

a) Direct Mapping

- ❖ The simplest technique, known as direct mapping, maps each block of main memory into only one possible cache line/block.
- ❖ The line number/cache block number of cache to which a particular main memory block can map is given by-

$$i = j \text{ modulo } m$$

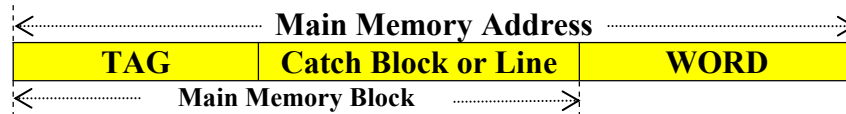
where
 i =cache line/block number
 j = main memory block number
 m =number of lines/block in the cache

Or

Cache line/block number
 = (Main Memory Block number) Modulo (Number of lines/blocks in Cache)

- ❖ The **drawback with direct mapped cache** is that it severely limits what data or instructions can be stored in the memory cache, so cache hits are rare. In other words, since more than one memory block is mapped onto a given cache block position, contention may arise for that position even when the cache is not full.
- ❖ **Need of Replacement Algorithm:** In direct mapping, there is no need of any replacement algorithm. This is because a main memory block can map only to a particular line of the cache. Thus, the new incoming block will always replace the existing block (if any) in that particular line.

- ❖ **Division of Physical Address:** Placement of a block in the cache is determined from the memory address. In direct mapping, the memory address is divided into three fields as shown below:



Where,

Word field represents number of bits required to identify a word in a block. If each block or Line contains 16 words, then $16=2^4 \Rightarrow 4$ bits, the word field will contain 4bits. If each block or Line contains 64 words, then $64=2^6 \Rightarrow 6$ bits.

Catch Block or Line field represents number of bits required to identify a catch block or line in catch memory. When a new block enters the cache, the bits in the cache field determinesthe cache position in which this block must be stored. If cache memory contains 2K words, then $2k=2 \cdot 2^{10}=2^{11} \Rightarrow 11$ bits are required to identify a cache block in this case.

Tag field represents the rest of the bits after subtracting word and cache block field bits from the main memory address bits issued by CPU. If the tag field contains t bits, then it identify which of the 2^t blocks that are mapped into this cache position are currently resident in the cache.

- ❖ **Example-1:** A computer system has a main memory consisting of 64K words.It also has a 2K word cache organised in directed-mapped cache manner, with 16 words per block. Calculate the number of bits in each of the TAG, BLOCK and WORD fields of the main memory address format. Also determine cache locations in which to store main memory blocks

Answer

Size of main memory=64K words= $2^6 \cdot 2^{10}=2^{16} \Rightarrow 16$ bits are required to identify one word of main memory. This is main memory address.

Size of cache memory=2K words

Number of words in a block= $16=2^4 \Rightarrow 4$ bits are required to identify one word within a given block. So the **WORD field** will be 4.

Number of blocks in main memory= $64K/16=2^{16}/2^4=2^{12}=4096 \Rightarrow$ requires 12 bits to identify a block in main memory.

Number of blocks in cache memory= $2K/16=2^{11}/2^4=2^7=128 \Rightarrow$ requires 7 bits to identify a block in cache memory. So the **BLOCK field** will be 7.

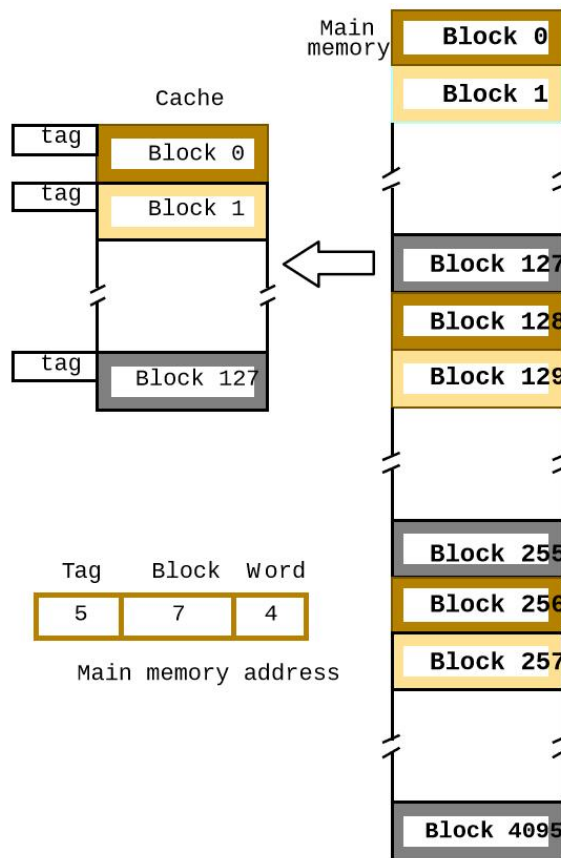
Now the **TAG filed** = **Main Memory address field** -**BLOCK field**-**WORD filed**
 $= 16-7-4= 5$ bits.

Tag	Catch Block or Line	Word
5	7	4

In this techniques, block j of main memory maps onto the block $i = j$ modulo 128 of the cache.

Cache Block (i) = j modulo 128	Main Memory Block (j)
-------------------------------------	-----------------------

0	0, 128, 256, 512,
1	1, 129, 257, 513,
2	
3	
.....	
.....	
.....	
126	
127	

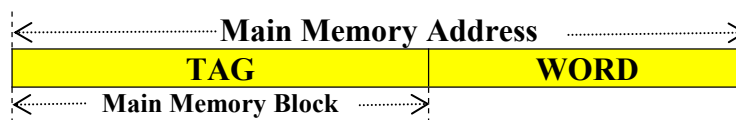


(Direct-Mapped Cache)

- ❖ **Explanation of the working of direct mapped cache:** After CPU generates a memory request,
 - ✧ The line number field of the address is used to access the particular line of the cache.
 - ✧ The tag field of the CPU address is then compared with the tag of the line.
 - ✧ If the two tags match, a cache hit occurs and the desired word is found in the cache.
 - ✧ If the two tags do not match, a cache miss occurs.
 - ✧ In case of a cache miss, the required word has to be brought from the main memory.
 - ✧ It is then stored in the cache together with the new tag replacing the previous one.

b) Associative mapping

- ❖ In **associative mapping**, a block of main memory can be **placed into any line of the cache** that is freely available at that moment. Cache has to replace (eject) an existing block only if the cache is full by using **replacement algorithm** like FCFS Algorithm, LRU Algorithm etc. Replacement algorithm suggests the block to be replaced if all the cache lines are occupied.
- ❖ To determine if a memory block is in the cache, each of the tags are simultaneously checked for a match. A search of this kind is called associative search.
- ❖ **Advantages:** It gives complete freedom in choosing the cache location in which to place the memory block. Thus, the space in the cache can be used more efficiently.
- ❖ **Dis-Advantages:** The cost of an associative cache is higher than the cost of a direct-mapped cache because the need to search all tag patterns to determine whether a given block is in the cache or not as More space is needed for the tag field. The most important disadvantage is the complex circuitry needed to examine all of the tags in parallel in the cache. A replacement algorithm must be used to determine which line of cache to swap out
- ❖ **Division of Main Memory Address:** In associative mapping, the memory address is divided into three fields as shown below:



Where,

WORD field represents number of bits required to identify a word in a block. If each block or Line contains 16 words, then $16=2^4 \Rightarrow 4$ bits, the word field will contain 4 bits.

Tag field represents the rest of the bits after subtracting word field bits from the main memory address bits issued by CPU.

Number of bits in TAG Field = Number of bits in Main Memory Address – Number of bits in WORD field offset

- ❖ **Example-1:** A computer system has a main memory consisting of 64K words. It also has a 2K word cache organised in associative-mapped cache manner, with 16 words per block. Calculate the number of bits in each of the TAG and WORD fields of the main memory address format.

Answer

Given Data,

Cache memory size = 2K words

Main Memory Block size = Cache Block Size = 16 words per block

Main memory size = 64K words.

Now,

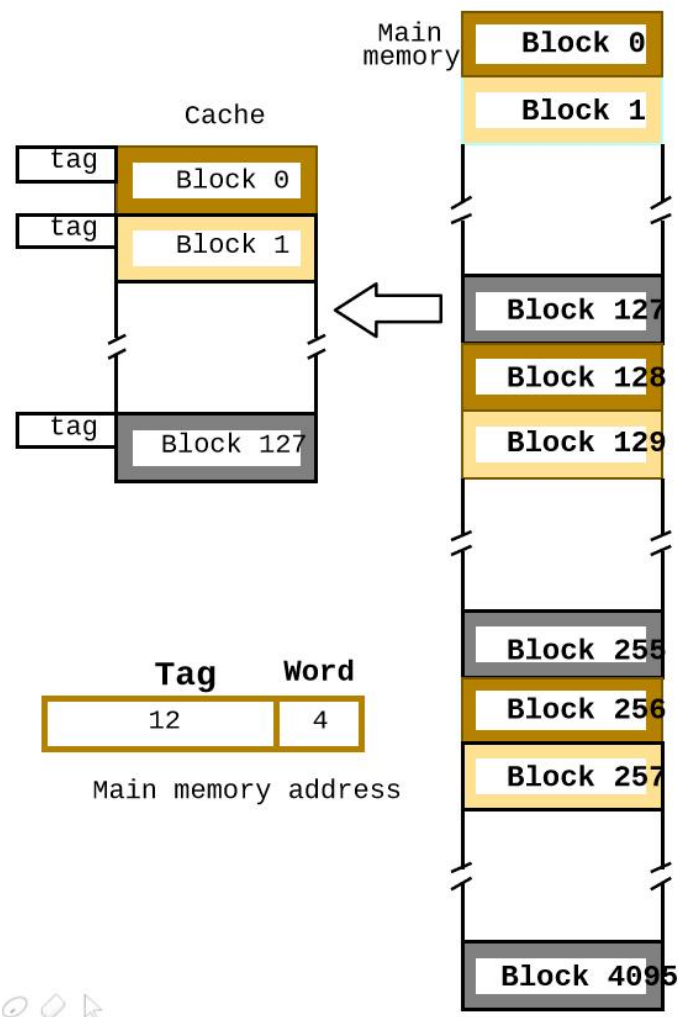
Size of main memory = 64K words = $2^6 \cdot 2^{10} = 2^{16} \Rightarrow 16$ bits are required to identify one word of main memory that is the number of bits in main memory address = 16 bits

As main memory block size is $16=2^4 \Rightarrow$ So 4 bits are required to identify a word in a given block.

Number of bits in **WORD** field = 4

Number bits in TAG field = Bits in Main memory Address - Number of bits in WORD field = $16 - 4 = 12$ bits





(Associative-Mapped Cache)

c) Set-Associative Mapping

- ❖ It utilizes the strengths of direct and associative mapping while trying to reduce their disadvantages.
- ❖ In Set-Associative mapping, blocks are grouped into sets and mapping allows a block of the main memory to reside in any block of a specified set. Hence, the contention problem of the direct method is erased by having a few choices for block placement. At the same time, the hardware cost is reduced by decreasing the size of the associative search.
- ❖ The cache is divided into v sets of k lines per set.

$$m = v \times k \Rightarrow v = m/k$$

$$i = j \text{ modulo } v$$

Where,

i = cache set number

j = main memory block number

m = number of lines in cache

v = number of sets

k = number of lines in each set

The set of the cache to which a particular block of the main memory can map is given by-

$$\text{Cache set number} = (\text{Main Memory Block Address}) \text{ Modulo } (\text{Number of sets in Cache})$$

- ❖ **Division of Main Memory Address:** The Set-Associative Mapping is called k-way Set-Associative Mapping, where each set contains k cache lines/block. In fully associative mapping, the physical address is divided as



Where,

WORD field represents number of bits required to identify a word in a block. If each block or Line contains 16 words, then $16=2^4 \Rightarrow 4$ bits, the word field will contain 4 bits.

SET field represents number of bits required to identify a set in cache memory.

- ❖ N.B. 1-way Set Associative Mapping $\equiv$ Direct Mapping
- ❖ **Example-1:** A computer system has a main memory consisting of 64K words. It also has a 2K word cache organised in associative-mapped cache manner, with 2 blocks per set and 16 words per block. Calculate the number of bits in each of the TAG and WORD fields of the main memory address format.

Answer

Given Data,

Cache memory size = 2K words

Main Memory Block size = Cache Block Size = 16 words per block

Set Size = 2 Cache blocks or lines per set

Main memory size = 64K words.

Now,

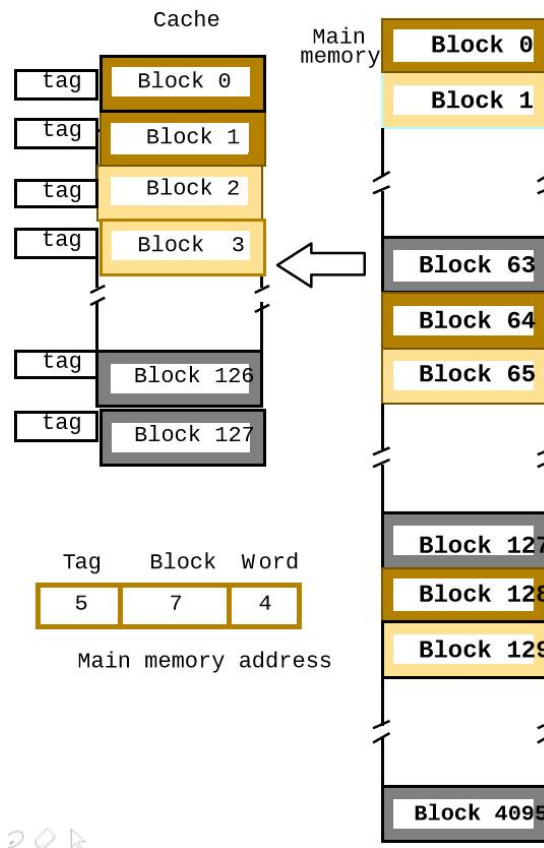
Cache block size = 16 $\Rightarrow \log_2(16) = 4$ bits are required to identify a word in a block. This is the **WORD field**.

Number of cache block or line = $2K/16 = (2 \cdot 2^{10})/(2^4) = 2^{11}/2^4 = 2^7 = 128$ blocks

Number of set = $128/2 = 64 \Rightarrow \log_2(64) = 6$ bits are required to identify a set. This is the **SET field**.

Now the **TAG field** = Main memory address bits - WORD field - SET field = $16 - 4 - 6 = 6$ bits.

TAG	SET	WORD
6	6	4



(2 way Set-Associative-Mapped Cache)

4.7 Solved Problems (Imp)

Q1. A computer system has a main memory consisting of 1M 16-bit words. It also has a 4K word cache organised in directed-mapped cache manner, with 64 words per block. Calculate the number of bits in each of the TAG, BLOCK and WORD fields of the main memory address format.

Solution

Size of main memory = 1M words = 2^{20} $\Rightarrow$ 20 bits are required to identify one word of main memory.

Size of cache memory = 4K words

Number of words in a block = 64 = 2^6 $\Rightarrow$ 6 bits are required to identify one word within a given block. So the **WORD field** will be 6.

Number of blocks in cache memory = $4K/64 = (2^2 \cdot 2^{10})/2^6 = 2^{12}/2^6 = 2^6$ $\Rightarrow$ requires 6 bits to identify a block in cache memory. So the **BLOCK field** will be 6.

Now the **TAG field** = Main Memory address field - BLOCK field - WORD field
 $= 20 - 6 - 6 = 8$ bits.

Tag	Catch Block or Line	Word
8	6	6

Q2. A computer that has an instruction cache organised in the direct-mapped manner and that has the following parameters:

Main memory size : 64K words

Cache size : 1K words

Block size : 128 words

Specify the number of bits in the TAG, BLOCK and WORD fields in main memory address. (Ref. Carl Hamacher CO-BOOK-5th.Ed.Exercise-5.6.a problem/Page-361)

Solution

Size of main memory=64K words= $2^6 \cdot 2^{10} = 2^{16} \Rightarrow$ 16 bits are required to identify one word of main memory. Main memory Address=16 bits

Size of cache memory=1K words

Number of words in a block=128= $2^7 \Rightarrow$ 7 bits are required to identify one word within a given block. So the **WORD field** will be 7.

Number of blocks in cache memory=1K/128= $2^{10}/2^7 = 2^3 \Rightarrow$ requires 3 bits to identify a block in cache memory. So the **BLOCK field** will be 3.

Now the **TAG field** = Main Memory address field -BLOCK field-WORD field
= 16-3-7=6 bits.

Tag	Catch Block or Line	Word
6	3	7

Q3. A computer system has a main memory consisting of 1M 16-bit words.It also has a 4K word cache organised in associative-mapped cache manner, with 64 words per block. Calculate the number of bits in each of the TAG and WORD fields of the main memory address format.

Solution

Size of main memory=1M words= $2^{20} \Rightarrow$ 20 bits are required to identify one word of main memory.

Size of cache memory=4K words

Number of words in a block=64= $2^6 \Rightarrow$ 6 bits are required to identify one word within a given block. So the **WORD field** will be 6.

Number bits in TAG field = Bits in Main memory Address - Number of bits in WORD field = 20 -6 =14 bits

TAG	WORD
14	6

Q4. A block-set-associative cache consists of a total of 64 blocks divided into 4 block sets. The main memory contains 4096 blocks, each containing of 128 words.

- a) How many bits are there in a main memory address?
- b) How many bits are there in each of the TAG, SET and WORD fields?

(Ref. Carl Hamacher CO-BOOK-5th.Ed.Exercise-5.9 problem/Page-363)

Solution

Given Data,

The main memory contains 4096 blocks, each containing of 128 words. So the size of main memory = $4096 \times 128 \text{ words} = 2^{12} \cdot 2^7 = 2^{19} \Rightarrow$ The main memory address = 19 bits

Main Memory Block size = Cache Block Size = 128 words per block = $2^7 \Rightarrow$ The **WORD field** = 7

Number of blocks in cache = 64 blocks

4 block sets $\Rightarrow$ Set Size = 4 Cache blocks or lines per set

So, Number of sets = $64/4 = 16 = 2^4 \Rightarrow$ **SET Field** = 4

Now the **TAG field** = Main memory address bits - WORD field - SET field = $20 - 7 - 4 = 9$ bits.

TAG	SET	WORD
8	4	7

Q5. A computer system has a main memory consisting of 1M 16-bit words. It also has a 4K word cache organised in the block-set-associative manner, with 4 blocks per set and 64 words per block. Calculate the number of bits in each of the TAG, SET and WORD fields of the main memory address format. (Ref. Carl Hamacher CO-BOOK-5th.Ed.Exercise-5.10.a problem/Page-363)

Solution

Given Data,

The main memory contains 1M words = $2^{20} \Rightarrow$ The main memory address = 20 bits

Main Memory Block size = Cache Block Size = 64 words per block = $2^6 \Rightarrow$ The **WORD field** = 6

Cache size = 4K words

Number of blocks in cache = $4K/64 = 2^{12}/2^6 = 2^6$ blocks

4 block sets $\Rightarrow$ Set Size = 4 Cache blocks or lines per set

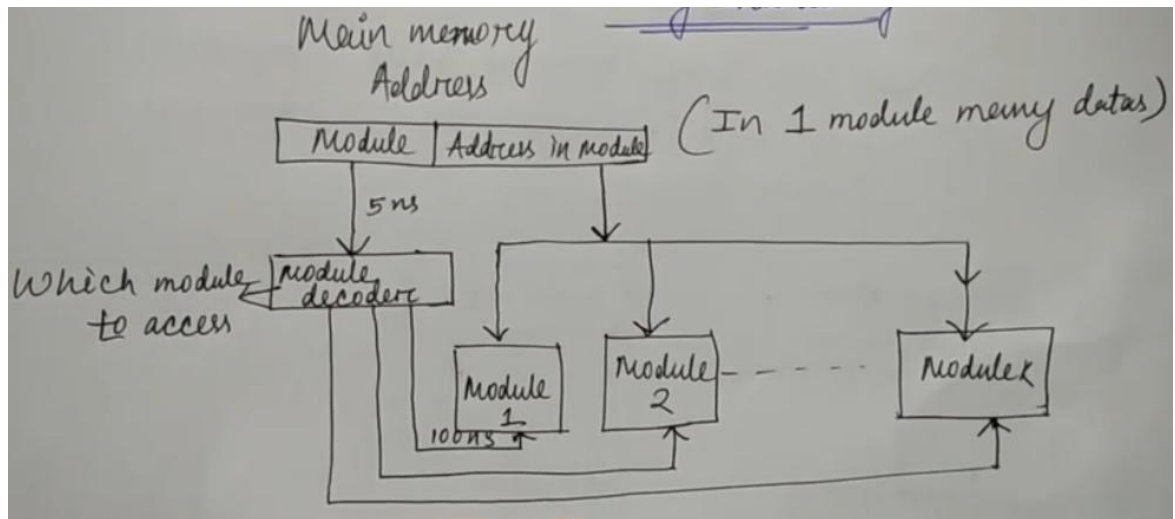
So, Number of sets = Number of blocks in cache / blocks per set = $2^6/4 = 2^4 \Rightarrow$ **SET Field** = 4

Now the **TAG field** = Main memory address bits - WORD field - SET field = $20 - 6 - 4 = 10$ bits.

TAG	SET	WORD
10	4	6

5 Memory Interleaving

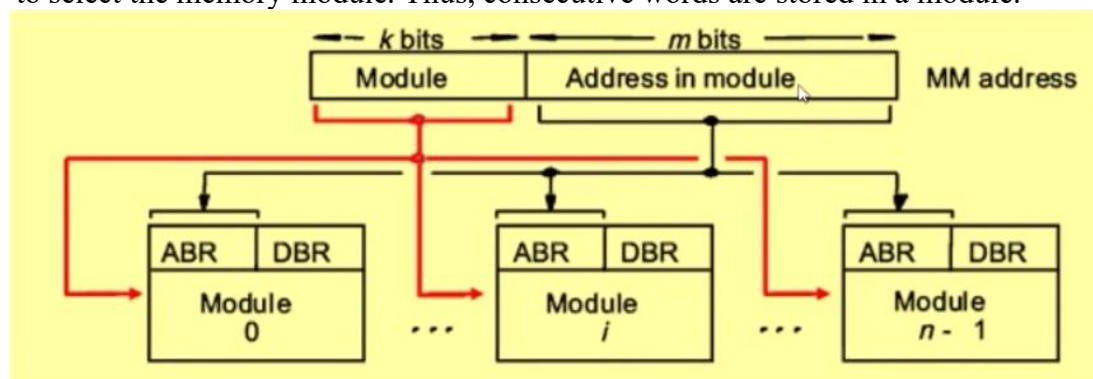
- A single memory module causes sequential access (only one memory access performed at a time). It is not efficient.
- **Memory Interleaving** is a technique which divides memory into a number of modules such that successive words in the address space are placed in the different module and memory requests may be issued to all modules at the same time/simultaneously. Thus, the aggregate rate of transmission of words to and from memory system can be increased. So It is efficient.
- But, how to divide the main memory address and distribute the main memory address space into different modules to achieve the above parallelism. Let main memory address = $n = m + k$, where, k bits represents the module number and m bits represents the address in module.



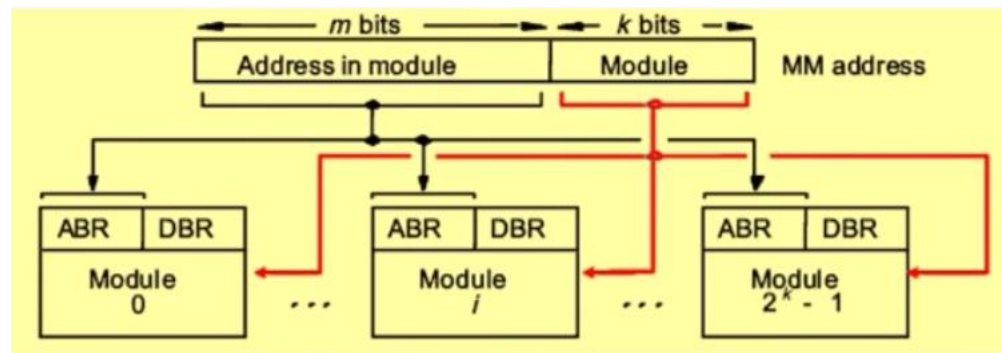
Let us assume that in order to decode a module (to identify a module), the address decoder takes 5ns and to access the word in a module it takes 100ns.

For three instructions, it will take $5 \times 3 + 100 = 115$ ns, but in normal process to access memory it takes $100 \times 3 = 300$ ns. So memory interleaving is better.

- **Types of Memory Interleaving:** Depending on the way to distribute consecutive words of main memory into different modules, there are two ways to achieve memory interleaving.
 - a) **High-order-interleaving (HOI):** Here, the high order bits of the address are used to select the memory module. Thus, consecutive words are stored in a module.



- b) **Low-order-interleaving (LOI):** Here, the low order bits of the address are used to select the memory module. Thus, consecutive words are stored in consecutive modules.



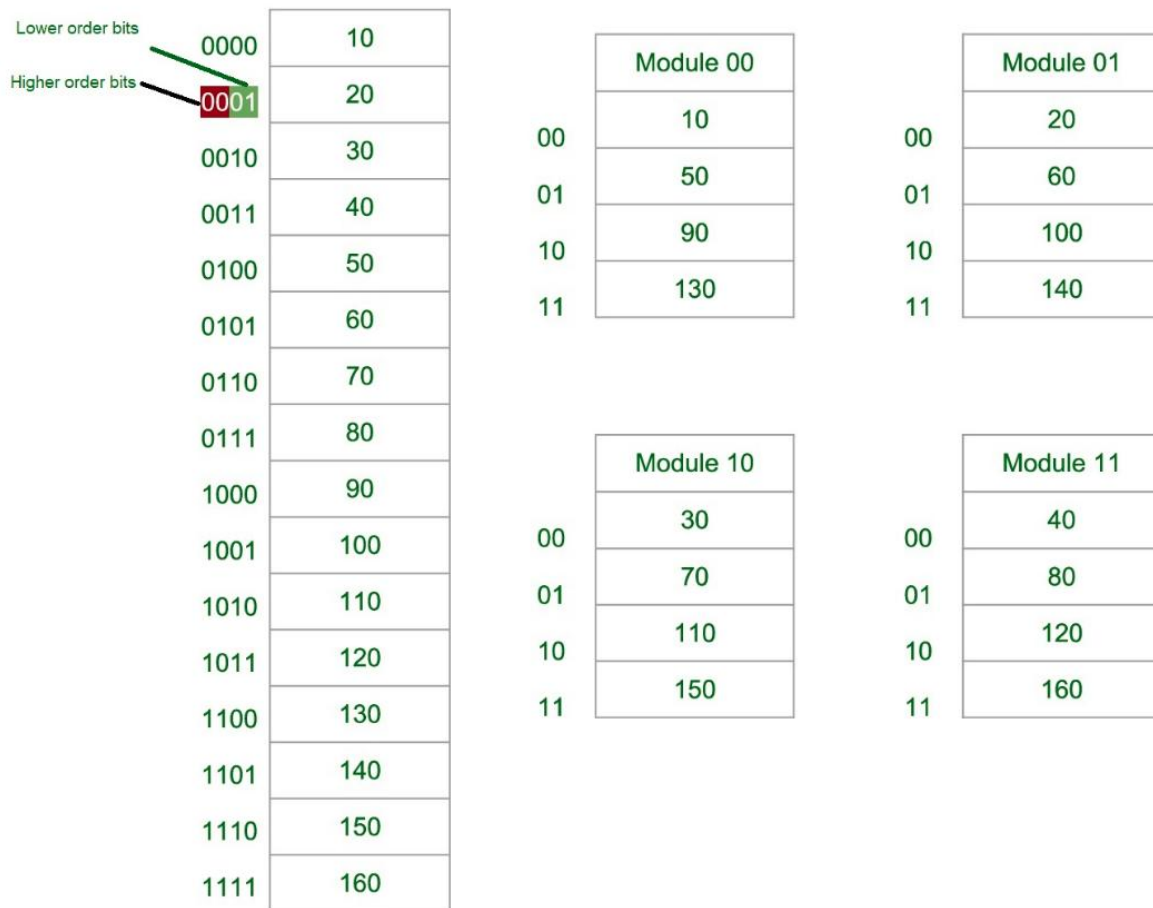
- Example:** let main memory contain $n=4$ bits. Thus it contains $2^4=16$ words. How to distribute these 16 words in different memory modules. Assume $k=2$, two bits are required to uniquely identify a memory modules. So there will be four memory (2^2) modules. And each module will contain four words (as $m=2$ means $2^2=4$ words). As per the above types of memory interleaving, how the main memory words are distributed into different modules are explained below.
- High-order-interleaving (HOI) where consecutive words are stored in a module.**

Lower order bits	0000	10				
Higher order bits	0001	20				
	0010	30				
	0011	40				
	0100	50				
	0101	60				
	0110	70				
	0111	80				
	1000	90				
	1001	100				
	1010	110				
	1011	120				
	1100	130				
	1101	140				
	1110	150				
	1111	160				

		Module 00			Module 01
00		10	00		50
01		20	01		60
10		30	10		70
11		40	11		80

		Module 10			Module 11
00		90	00		130
01		100	01		140
10		110	10		150
11		120	11		160

- **Low-order-interleaving (LOI) where consecutive words are stored in consecutive modules.**



5.1 Advantages of Memory Interleaving

- Whenever, processor request data from the main memory, a block (chunk) of data is transferred to the cache and then to processor. So whenever a cache miss occurs the data is to be fetched from main memory. But main memory is relatively slower than the cache. So to improve the access time of the main memory interleaving is used.

Any typo error/mistakes found mail to anil.swainfcs@kiit.ac.in or anilkumarswain@gmail.com

